Arquitectura y fusión sensorial de AiGB_UEx_Navigation

Localización del Husky A300, percepción RGB-D y seguimiento humano con LiDAR y cámara ZED

Informe técnico basado en el código y la configuración del repositorio

14 de octubre de 2026

Resumen

Este informe reconstruye el funcionamiento de la versión de trabajo del repositorio `AiGB_UEx_Navigation`. El sistema integra un robot Husky A300 simulado en Webots, drivers RoboComp, transporte de datos mediante DDS y agentes que comparten un grafo DSR. Hay tres mecanismos de fusión diferentes: un filtro de Kalman extendido que localiza el robot mediante GPS, odometría de ruedas y orientación/velocidad angular de la IMU; un seguimiento humano que combina esqueletos RGB-D con evidencia geométrica del LiDAR; y una rejilla de ocupación que integra rayos LiDAR y profundidad ZED para representar obstáculos. El código revisado no integra el LiDAR ni la odometría visual de la ZED en el EKF de localización. La fusión humana tampoco es un estimador conjunto de articulaciones y nube LiDAR: la cámara inicia las identidades y estima la postura, mientras el LiDAR contribuye a la existencia y al desplazamiento horizontal de personas previamente conocidas. Se documentan ecuaciones, marcos, parámetros, fuentes concretas y límites de la implementación. Dos comprobaciones aisladas existentes, del EKF y de la cinemática humana, se ejecutaron satisfactoriamente; no se realizó una campaña de navegación o percepción en vivo.

Índice

1. Alcance, método y trazabilidad 3

2. Visión general del sistema 3

3. Simulación, sensores y adquisición 5

4. Comunicaciones, unidades, calibración y tiempo 6

5. Localización: GPS, ruedas e IMU mediante EKF 8

6. Percepción visual: de RGB-D a personas 10

7. Modelo cinemático humano e inferencia 11

8. Fusión LiDAR–ZED para seguimiento humano 12

9. Fusión geométrica de obstáculos y rejilla residual 14

10. Cómo repercute la percepción en la navegación 15

11. Parámetros esenciales para reproducir el perfil 15

12. Limitaciones y hallazgos que deben figurar en el reporte 16

13. Validación realizada y evaluación propuesta 18

14. Construcción y ejecución del proyecto 19

15. Redacción sintética utilizable en un technical report 19

1. Alcance, método y trazabilidad

El objeto de estudio es el **árbol de trabajo presente**, con los perfiles `active_inference/utils/sub_husky.toml` y `active_inference/utils/cognitive_husky.toml` como referencia de ejecución. Se inspeccionaron los componentes, sus configuraciones, los productores y consumidores de cada dato y las funciones que efectivamente se invocan. La identificación de un algoritmo en un archivo o de una opción en TOML no se tomó por sí sola como prueba de que esa ruta se ejecute. [2, 3]

La referencia Git observada fue `afab2d8cc17628b015118e930d40922f21cf6da5`. El árbol tenía modificaciones y numerosos archivos eliminados localmente; por ello, ese identificador *no* describe por sí solo el contenido analizado. No se recuperaron componentes eliminados ni se alteró la implementación. Las referencias finales dan rutas relativas, funciones o intervalos de líneas del contenido revisado. Cuando un comentario y el flujo ejecutable difieren, este informe prioriza el segundo.

Se distinguen cuatro niveles de evidencia:

- **Implementado**: comportamiento visible en las funciones y llamadas revisadas.
- **Configurado**: valores del perfil Husky; no equivalen a una frecuencia o precisión medida.
- **Comprobado aisladamente**: resultados de pruebas ejecutadas durante esta revisión.
- **Pendiente de validación**: consecuencias plausibles del código, problemas potenciales y experimentos propuestos.

El README declara localización de aproximadamente 2 cm y navegación/seguimiento funcionales en simulación. Se conserva esa afirmación como **declaración del proyecto**, no como resultado experimental reproducido aquí. Los valores medidos durante esta revisión se presentan separadamente en la sección 13. [1]

2. Visión general del sistema

2.1. Organización del repositorio

Directorio	Responsabilidad
`webots-agri/`	Modelos PROTO del Husky, terreno, palmeras y manzanos; mundos de simulación; generadores de hileras y malla de autofiltro; controlador de autocomprobación.
`webots-bridge/`	Accede a los dispositivos Webots, adapta coordenadas y mensajes, ofrece interfaces Ice y aplica órdenes de velocidad a las ruedas. También expone el GPS.
`robocomp-robolab/`	Drivers conservados: LiDAR, ZED, IMU y mando Xbox. Los tres primeros adaptan las fuentes al plano de datos sensorial.
`active_inference/robot_concept/`	Carga el modelo del robot en DSR, incorpora telemetría y anuncia los descriptores de los flujos sensoriales.
`active_inference/openfield_concept/`	EKF de localización planar, marco `field`, límite del área de trabajo y publicación de pose/covarianza.
`active_inference/retina/`	Percepción visual: segmentación de instancias y pose humana; publica máscaras y esqueletos. Contiene otras rutas deshabilitadas en el perfil Husky.
`active_inference/human_concept/`	Asociación temporal, modelo cinemático por persona, seguimiento con LiDAR, existencia y nodos `person_*`.
`active_inference/residual_concept/`	Ocupación de obstáculos a partir de LiDAR y profundidad; resta explicaciones geométricas al leer la rejilla.

Directorio	Responsabilidad
`active_inference/controller/`	Planificación, seguimiento de ruta, tratamiento de obstáculos y mando de la base.
`active_inference/common/`	Transporte DDS, lectura temporal de transformaciones, existencia, geometría, huella, seguimiento y utilidades compartidas.
`active_inference/utils/`	Lanzadores, envío de objetivos y monitorización de procesos, comunicaciones y ventanas.

La arquitectura procede de una adaptación de componentes para interiores. Quedan nombres como `room`, `bpearl` o referencias a Shadow en comentarios y funciones, pero el perfil agrícola utiliza `field`, un único LiDAR Helios y la ZED frontal. `common/world_frame` prefiere el nodo de tipo `field` y conserva `room` como alternativa. También existe `husky_apple.wbt`, además de los mundos de palmeras descritos en el README; su presencia no demuestra que los parámetros de seguimiento estén validados entre árboles. [1, 4, 5]

2.2. Flujo completo

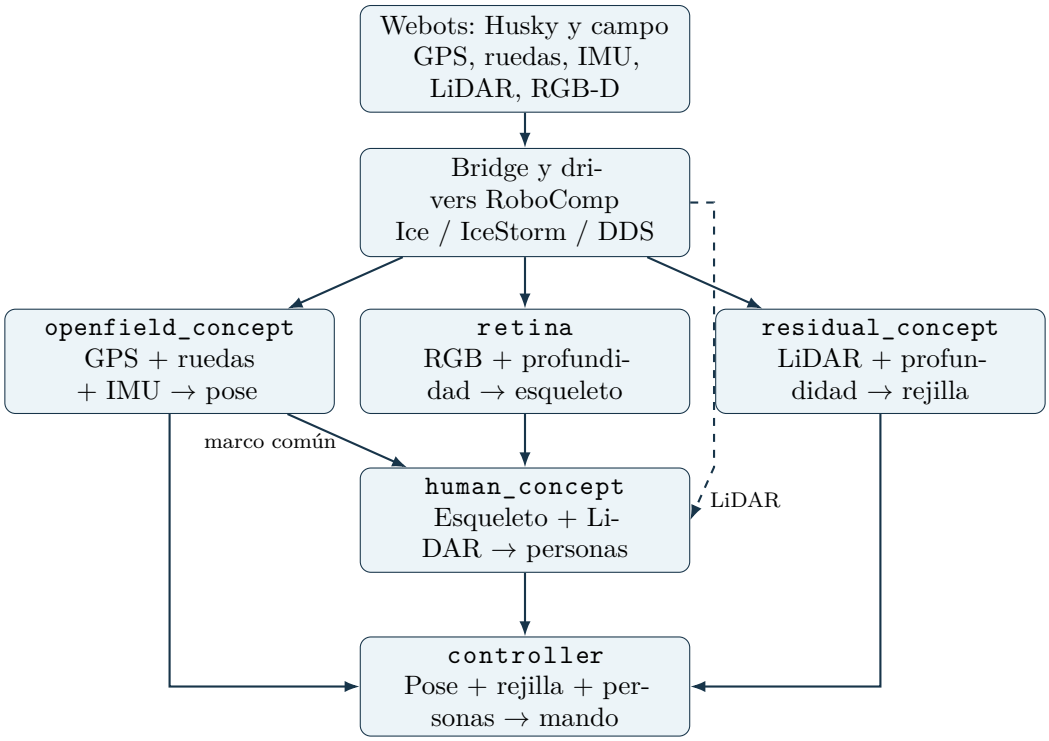


Figura 1: Dependencias funcionales principales. DSR contiene el estado compartido y las transformaciones; los datos sensoriales voluminosos circulan por el plano DDS. La pose también permite registrar la rejilla en el campo. El mando vuelve al bridge y a las ruedas.

`robot_concept` constituye la base común del grafo. El lanzador cognitivo enumera después `openfield_concept`, `residual`, `controller`, `retina` y `human_concept`. El orden de la lista facilita el arranque, pero no es sincronización de sensores: los agentes verifican disponibilidad del grafo, de productores y de datos mediante sus ciclos y mecanismos de presencia. [3, 9]

2.3. Qué significa «fusión» en este repositorio

Problema	Entradas	Estado estimado	Mecanismo
Localización	GPS, ruedas, IMU	x, y, θ del robot	EKF: predicción y correcciones.
Postura humana	RGB y profundidad ZED	Articulaciones y alineamiento del cuerpo	YOLO-pose, desproyección y ajuste cinemático.
Seguimiento humano	Esqueleto y LiDAR	Identidad, centro horizontal y existencia	Asociación geométrica, relevo de posición y evidencia en log-odds.
Obstáculos	LiDAR y profundidad ZED	Ocupación del campo	Integración de rayos en una rejilla común.

La pose del robot es una **entrada geométrica** de la percepción: permite situar mediciones tomadas desde sensores móviles en el campo. No se encontró en el camino activo una corrección recíproca del EKF basada en personas, máscaras o alineamiento LiDAR. Por tanto, el sistema no debe describirse como SLAM LiDAR–visual–inercial ni como un único filtro que fusiona todos los sensores. [24, 26, 40, 44]

3. Simulación, sensores y adquisición

3.1. Robot y escenarios

El Husky tiene cuatro ruedas accionadas con cinemática *skid-steer*; el giro implica deslizamiento lateral. El modelo usa radio de rueda de 0.1651 m y semiancho de vía de 0.2854 m en el bridge, con `Base.TrackScale=1.875`. Este multiplicador modifica la relación entre diferencia de velocidades de ruedas y giro; no elimina la variación del deslizamiento con la maniobra. La elección explica que el EKF use el giróscopo para predecir orientación. [6, 7, 24]

El mundo de campo declara paso básico de 8 ms, referencia GPS $(38.8794°, -6.9707°, 186\,\text{m})$ y norte en $+Y$. Dos peatones recorren trayectorias predefinidas, a 0.5 y 0.6 m/s respectivamente. El archivo indica que están destinados a percepción y carecen de objeto de colisión física. Por ello, ver al robot atravesar o evitar gráficamente a un trabajador no constituye por sí solo una prueba física de contacto o seguridad. [5]

3.2. Especificación realmente utilizada

Sensor	Configuración observada
LiDAR Helios	Montaje $(0, -0.25, 0.95)$ m respecto a `body`. El PROTO genera 900 muestras horizontales y 50 capas sobre aproximadamente 110° verticales, alcance 30 m y ruido nominal 0.02 m. El bridge conserva elevaciones entre −55° y +15°: se retienen 32 capas, aproximadamente de +14.6° a −55°. No deben confundirse las 50 capas generadas con las 32 publicadas tras el recorte.
ZED simulada	Montaje $(0, 0.42, 0.55)$ m, inclinación frontal descendente de aproximadamente 15°. Imagen 1280×720, campo horizontal de unos 110°. Profundidad mediante `RangeFinder`, límites 0.3–20 m.
GPS	Antena $(0, -0.25, 1.02)$ m. Ruido configurado por eje de 0.02 m, correlación 0.9; actualización solicitada cada 100 ms.

Sensor	Configuración observada
IMU	Montaje $(0, 0, 0.2251)$ m en el grafo. El bridge lee giróscopo, acelerómetro y orientación de `InertialUnit`; el EKF utiliza ω_z y yaw.

Fuentes: PROTO, modelo JSON del robot y configuración del bridge. Los valores son propiedades del escenario revisado, no especificaciones garantizadas de equipos reales. [6, 8, 7]

Hay una discrepancia concreta: el descriptor configurado en el driver LiDAR conserva `RangeMax M=100.0` y referencias textuales a Shadow, mientras el PROTO Husky limita el sensor a 30 m. Esta diferencia afecta a la interpretación de metadatos; el alcance efectivo simulado procede del dispositivo. [14, 6]

3.3. La profundidad simulada no se calcula por correspondencia estéreo

La ruta Husky usa `Config.Simulated=true`. Webots produce una imagen de `Camera` y una matriz de profundidad de `RangeFinder`; el bridge las empaqueta conjuntamente, y `zed_camera` obtiene ese paquete mediante `CameraRGBDSimple::getAll`. Esta ruta no reconstruye profundidad a partir de dos imágenes ni ejecuta la estimación estéreo del SDK ZED. Es una **emulación RGB-D de la interfaz ZED**. [11, 17, 16]

El driver contiene otra rama para cámara real, con SDK ZED, profundidad `NEURAL_PLUS` y seguimiento posicional. Sin embargo, la llamada que publicaría la pose está comentada en `process_pose_data`. Además, la configuración activa es simulada. La existencia de `enablePositionalTracking` en el driver no permite afirmar que la odometría visual ZED localice el Husky en este perfil. [17]

4. Comunicaciones, unidades, calibración y tiempo

4.1. Dos planos de información

El plano DSR, configurado en dominio 0, representa robot, sensores, campo, personas, rejilla y atributos. Los mensajes voluminosos se transmiten por DDS en dominio 7: `rc/zed/rgb`, `rc/zed/depth`, `rc/lidar3d/points` y `rc/imu/data`. Los drivers anuncian descriptores; `robot_concept` puede retransmitirlos al grafo, y los consumidores descubren dominio, tópico y parámetros del flujo a partir del nodo correspondiente. Los perfiles solicitan memoria compartida y *data sharing*. Esto reduce copias en el transporte, pero no implica que no existan copias en el procesamiento de imágenes. [9, 19, 14, 16]

Ice se utiliza, entre otras cosas, para obtener RGB-D simulado, consultar el GPS y mandar velocidades. IceStorm transporta publicaciones como la telemetría `FullPoseEstimationPub`; `robot_concept` extrae de ella las velocidades de ruedas y guarda aparte la pose de referencia del simulador. [7, 10, 2]

4.2. Marcos de referencia y extrínsecas

Se adopta la notación ${}^{A}T_{B}$ para la transformación que lleva coordenadas del marco B al marco A. El marco corporal usa X hacia la derecha, Y hacia delante y Z hacia arriba. El campo usa este, norte y arriba. La cámara en los consumidores también usa Y como eje de profundidad; no se debe aplicar sin conversión una fórmula de cámara con Z óptico. [25, 8, 31]

El archivo `active_inference/robot_concept/husky.json` describe `root->husky->body->se`

nsores. El enlace husky->body es identidad. Las extrínsecas principales son:

$$ {}^{B}T_{L} = \begin{bmatrix} I & (0, -0.25, 0.95)^{T} \\ 0 & 1 \end{bmatrix}, \tag{1} $$

$$ {}^{B}T_{C} = \begin{bmatrix} R_{x}(-0.2618) & (0, 0.42, 0.55)^{T} \\ 0 & 1 \end{bmatrix}. \tag{2} $$

El bridge y el driver convierten previamente los ejes nativos de Webots. Por eso, copiar además la rotación de 90° del dispositivo Webots en la arista DSR del LiDAR duplicaría una conversión. El driver mantiene los puntos en el marco sensorial normalizado y deja el montaje corporal en la cadena RT. [8, 15, 6]

Para comparar geometrías tomadas en instantes distintos se busca:

$$ p_{L}^{F} = {}^{F}T_{B}(t_{L}) {}^{B}T_{L} p^{L}, \qquad p_{C}^{F} = {}^{F}T_{B}(t_{C}) {}^{B}T_{C} p^{C}. \tag{3} $$

La comparación cámara–LiDAR se efectúa así en un marco común. En la ruta humana no se encontró una proyección de todos los puntos LiDAR sobre las articulaciones 2D ni una calibración extrínseca optimizada en línea: se usan los montajes declarados. [20, 34, 40]

4.3. Unidades y autofiltro

El driver LiDAR convierte la representación interna en milímetros a metros antes de publicar DDS. El autofiltro de malla se aplica antes de publicar; usa la geometría del Husky y su montaje para retirar retornos del propio robot. Esa transformación para el filtrado no significa que el flujo publicado quede en coordenadas corporales. [15, 14]

La profundidad simulada se transmite como `float32` en metros. Retina interpreta `FORMAT_DEPTH_F32` directamente y convierte `Z16` de milímetros a metros. Debe conservarse esta distinción al conectar hardware real: en la rama SDK revisada se configura `sl::UNIT::MILLIMETER`, pero la publicación DDS copia los flotantes como `FORMAT_DEPTH_F32` sin conversión visible. Es una **incompatibilidad potencial de unidades en la ruta real**, no un fallo observado en la simulación. [17, 18, 21, 47]

4.4. Sincronización efectiva y límites

Retina empareja RGB y profundidad mediante `ZedFrameAligner`; el consumidor solicita profundidad con una tolerancia de 20 ms respecto al RGB más reciente. El procesamiento sigue una política de conservar el dato más nuevo: si el cálculo no alcanza la frecuencia de adquisición, puede saltarse imágenes para limitar el retraso. [22, 21, 23]

El lector LiDAR común consulta transformaciones interpoladas al sello temporal del barrido. El consumidor de esqueletos consulta `field<-zed` al instante publicado en el nodo `skeleton`. No hay una barrera global que espere una pareja LiDAR–cámara exacta: los agentes actualizan sus estados asincrónicamente. [20, 34]

Se emplean tres nociones temporales: reloj de simulación para integrar velocidades simuladas, época en milisegundos para registrar sensores/RT, y reloj monótono para caducidades de personas. El EKF usa `sim_stamp_ms` cuando existe. Sin esa distinción, una simulación que avanza más despacio que el tiempo real produciría desplazamientos incorrectos. La asociación de personas y la banda de movimiento, en cambio, usan tiempo monótono del proceso; su sensibilidad al factor de tiempo real merece una prueba específica. [26, 41, 34]

5. Localización: GPS, ruedas e IMU mediante EKF

5.1. Estado y entradas

`OpenFieldEkf` mantiene

$$\boldsymbol{x} = [x\ y\ \theta]^T, \qquad P \in \mathbb{R}^{3\times 3}. \tag{4}$$

Se trata de una pose planar: no estima altura, roll, pitch, velocidad como estado, sesgo de giróscopo ni escalas de ruedas. La velocidad longitudinal v_a, lateral v_s y angular ω son entradas. La orientación θ gira el eje corporal $+X$ respecto al este, de modo que el vector hacia delante es $[-\sin\theta, \cos\theta]^T$. [24, 25]

`read_odometry` lee velocidades publicadas por `robot_concept`. `pump_imu` predice con ω_z del giróscopo; la velocidad de giro de ruedas se conserva como lectura/diagnóstico, pero no se usa en la propagación del EKF. El acelerómetro tampoco entra en este filtro. [26, 10]

5.2. Predicción

Con $\bar{\theta} = \theta_k + \omega_k \Delta t/2$, la implementación usa:

$$\Delta x = (\cos\bar{\theta}\, v_s - \sin\bar{\theta}\, v_a)\Delta t, \tag{5}$$

$$\Delta y = (\sin\bar{\theta}\, v_s + \cos\bar{\theta}\, v_a)\Delta t, \tag{6}$$

$$\boldsymbol{x}^-_{k+1} = [x_k + \Delta x,\ y_k + \Delta y,\ \mathrm{wrap}_{[-\pi,\pi)}(\theta_k + \omega_k \Delta t)]^T. \tag{7}$$

El Jacobiano implementado y la propagación de incertidumbre son:

$$F = \begin{bmatrix} 1 & 0 & -\Delta y \\ 0 & 1 & \Delta x \\ 0 & 0 & 1 \end{bmatrix}, \qquad P^- = FPF^T + GQG^T, \tag{8}$$

$$G = \begin{bmatrix} -\sin\bar{\theta}\Delta t & \cos\bar{\theta}\Delta t & 0 \\ \cos\bar{\theta}\Delta t & \sin\bar{\theta}\Delta t & 0 \\ 0 & 0 & \Delta t \end{bmatrix}, \quad Q = \mathrm{diag}(\sigma_a^2, \sigma_s^2, \sigma_\omega^2)/\Delta t. \tag{9}$$

Las desviaciones son densidades dependientes del movimiento:

$$\sigma_a = \sqrt{0.01^2 + (0.05|v_a|)^2}, \tag{10}$$

$$\sigma_s = \sqrt{0.005^2 + (0.10|v_a\omega|)^2}, \tag{11}$$

$$\sigma_\omega = \sqrt{0.002^2 + (0.01|\omega|)^2}. \tag{12}$$

Así, GQG^T crece proporcionalmente a Δt, y el ruido lateral aumenta en maniobras que combinan avance y giro. Son coeficientes configurados; no se halló en esta revisión una calibración experimental nueva que los identificase. [24, 27]

5.3. GPS y brazo de palanca

El GPS observa la antena, no el origen del robot. Para $\boldsymbol{\ell} = (0, -0.25)^T$ m, el modelo de observación es:

$$h_G(\boldsymbol{x}) = \begin{bmatrix} x \\ y \end{bmatrix} + R(\theta)\boldsymbol{\ell}, \qquad H_G = \begin{bmatrix} I_2 & \dfrac{\partial R(\theta)\boldsymbol{\ell}}{\partial\theta} \end{bmatrix}. \tag{13}$$

El brazo de palanca acopla orientación y posición: una rotación desplaza la antena aunque el origen corporal permanezca fijo. La inicialización resta el brazo rotado al primer GPS disponible y propaga la incertidumbre del yaw a la posición y a sus covarianzas cruzadas. [24]

Con $R_G = 0.05^2 I_2$, se calcula innovación, ganancia y corrección:

$$\boldsymbol{r} = \boldsymbol{z}_G - h_G(\boldsymbol{x}^-), \qquad S = H_G P^- H_G^T + R_G, \tag{14}$$

$$K = P^- H_G^T S^{-1}, \qquad \boldsymbol{x}^+ = \boldsymbol{x}^- + K\boldsymbol{r}. \tag{15}$$

La covarianza se actualiza en forma de Joseph:

$$P^+ = (I - KH_G)P^-(I - KH_G)^T + KR_GK^T. \tag{16}$$

La distancia de Mahalanobis se devuelve como diagnóstico; no se utiliza como rechazo estadístico de GPS atípicos. El valor configurado de 0.05 m es superior al ruido nominal del dispositivo simulado para compensar parcialmente la correlación de sus muestras. No se estima un estado explícito de ruido GPS correlacionado. [24, 27]

El bridge convierte WGS84 a UTM y resta la referencia del mundo para entregar coordenadas locales. Aunque el proyecto las denomina ENU, la operación concreta implementada es una diferencia de coordenadas UTM; no debe confundirse con una conversión geodésica general mediante ECEF y plano tangente local. [12]

5.4. Corrección de orientación

La IMU proporciona una medición absoluta de yaw:

$$r_\theta = \mathrm{wrap}_{[-\pi,\pi)}(z_\theta - \theta^-), \quad H_\theta = [0\ 0\ 1], \quad R_\theta = 0.005^2. \tag{17}$$

La actualización es escalar y también usa forma de Joseph. El perfil limita estas correcciones a una cada 100 ms de reloj sensorial; la predicción continúa con cada IMU nueva. El objetivo es evitar tratar todas las orientaciones de alta frecuencia como observaciones independientes. [24, 26, 27]

En simulación, el yaw procede de `InertialUnit`; no es una implementación propia de fusión acelerómetro–giróscopo–magnetómetro. En un robot real será necesario definir qué estimador y qué referencia absoluta suministran ese yaw. [13]

5.5. Publicación y dependencias

Cada ciclo del agente lee odometría, procesa IMU, consulta GPS y publica. El período configurado es 20 ms; el GPS se consulta cada 50 ms, aunque su dispositivo se actualiza cada 100 ms. La deduplicación del GPS compara igualdad exacta de las coordenadas, ya que la interfaz consultada no aporta sello temporal del fix. Esta estrategia evita reutilizar un valor repetido, pero no distingue fixes diferentes con iguales coordenadas. [26, 27]

La arista almacenada tiene padre robot y destino `field`; contiene la inversa de la pose estimada:

$${}^{B}T_F = \begin{bmatrix} R^T & -R^T\boldsymbol{t} \\ 0 & 1 \end{bmatrix}. \tag{18}$$

El código transforma la covarianza con el Jacobiano de inversión y la inserta en los índices $x, y,$ yaw de una matriz 6×6. Publica también velocidad y su incertidumbre. El resto de agentes pide la transformación en el sentido que necesita mediante la API RT. Los ceros en las otras dimensiones expresan aquí una representación planar; no son una medición de precisión perfecta en altura o inclinación. [28]

La referencia del supervisor se usa para visualizar y registrar el error, buscando la estimación correspondiente al timestamp de esa referencia. No entra en las correcciones del EKF. Desactivar GPS en el visor deja navegación por integración de ruedas/giróscopo con crecimiento de incertidumbre; desactivar yaw elimina la corrección absoluta de orientación. [26, 10]

6. Percepción visual: de RGB-D a personas

6.1. Segmentación y pose son ramas diferentes

Retina ejecuta sobre RGB una etapa de segmentación configurada con `yolo26x-seg.onnx` y una etapa de pose con `yolo26l-pose.onnx`. Son nombres de modelos solicitados por el perfil; no se certifican aquí sus pesos ni su rendimiento. La segmentación acepta `person` y otras clases, con confianza mínima 0.25 e IoU de supresión 0.45. La pose humana usa confianza mínima de persona 0.30, confianza mínima de articulación 0.30 y decimación 1. [29, 30]

Las máscaras se publican en `masks`; los esqueletos en `skeleton`. La ruta de `human_concept` consume este último. No se encontró que la máscara de segmentación `person` sea requisito de aceptación del esqueleto o que se use para seleccionar sus muestras de profundidad. Por tanto, la fusión humana debe describirse como YOLO-pose más profundidad, no como una validación obligatoria entre dos detectores. [31, 34]

6.2. Reconstrucción 3D de las articulaciones

Para cada articulación 2D (u, v) aceptada se toman profundidades válidas de una ventana de semiancho 2 píxeles, es decir, 5×5. El código usa el elemento de orden central del conjunto de profundidades, una mediana robusta a valores extremos. Con d esa profundidad, la desproyección es:

$$\boldsymbol{p}^C = \left[\frac{(u - c_x)d}{f_x},\ d,\ \frac{(c_y - v)d}{f_y} \right]^T, \qquad c_x = W/2, \quad c_y = H/2. \tag{19}$$

Las focales proceden del flujo RGB-D. Un keypoint sin profundidad positiva y finita, fuera de imagen o con confianza insuficiente se publica como NaN y confianza cero. La ventana no está condicionada por una máscara humana: en un brazo delgado o un borde puede incluir fondo, incluso si la articulación 2D es correcta. [31]

Los 17 puntos COCO se reordenan al formato BODY_18. Se añade el cuello como punto medio tridimensional de los hombros cuando ambos son válidos; su confianza es la mínima de las dos. El nodo `skeleton` incluye contador de publicación, timestamp, número de cuerpos, IDs, 18×3 coordenadas por cuerpo y 18 confianzas. Las confianzas se transportan en $[0, 1]$ y se convierten a $[0, 100]$ en el consumidor. [31, 34]

6.3. Registro en el campo y asociación temporal

`DsrSkeletonSource::poll` transforma los esqueletos de cámara a campo y `assign_track_ids` asigna identidades persistentes según proximidad espacial. El centro se obtiene primero de las dos caderas; si no están disponibles, del cuello; finalmente, de la media de articulaciones válidas. Para una pista con edad Δt, la distancia máxima de asociación es:

$$g_{\text{cam}} = 0.35 + 2.0\Delta t \quad [\text{m}]. \tag{20}$$

Se ordenan los pares admisibles por distancia y se asignan vorazmente sin reutilizar un cuerpo ni una pista. Las pistas sin actualización durante 3 s se retiran. Un cuerpo sin asociación a menos de 0.5 m de otro ya asignado puede eliminarse como duplicado. [34]

Este mecanismo es asociación geométrica, sin descriptor de apariencia ni reidentificación aprendida. La exclusión uno a uno evita compartir un track en un frame, pero no garantiza conservar la identidad al cruzarse trabajadores, durante oclusiones largas o cuando sus centros se confunden. [34]

7. Modelo cinemático humano e inferencia

7.1. Qué estima el modelo

Cada `person_*` mantiene un modelo BODY_18 con 11 grados de libertad: tres ángulos por hombro, una flexión por codo y tres parámetros de cuerpo inferior (lb_x, lb_z, lb_{roll}). No es un modelo de articulación completa de las piernas. El modelo canónico usa Y vertical y se alinea al mundo mediante Kabsch; este marco interno no debe confundirse con el marco corporal del robot, donde Z es vertical. [37]

Para cada persona, las longitudes de segmentos se calibran con las observaciones válidas: la primera medición inicializa y las siguientes se incorporan mediante media exponencial con peso 0.05. La estimación de postura requiere al menos tres articulaciones utilizables para observar; el núcleo necesita además al menos tres anclajes válidos para el alineamiento. Los anclajes configurados son cuello, hombros y caderas. `MinValid=12` se carga en configuración y sus comentarios lo relacionan con acciones de observación, pero no se encontró un uso ejecutable de ese campo en el seguimiento revisado; no es el mínimo efectivo del ajuste. [36, 35, 38]

7.2. Ajuste por precisión y aproximación de Laplace

El estimador mantiene una media $\boldsymbol{\mu} \in \mathbb{R}^{11}$ y una precisión aproximada Λ. Una expresión explicativa de su objetivo es:

$$\mathcal{F}(\boldsymbol{\theta}) = \frac{1}{2M} \sum_{j \in \mathcal{V}} \pi_j \| R(\boldsymbol{\theta}) f_j(\boldsymbol{\theta}) + \boldsymbol{t}(\boldsymbol{\theta}) - \boldsymbol{y}_j \|^2 + \frac{1}{2} \| \boldsymbol{\theta} - \boldsymbol{\mu}_{prev} \|^2_{\Lambda_{dyn}} + \mathcal{P}(\boldsymbol{\theta}). \tag{21}$$

f_j es la cinemática directa, $R, \boldsymbol{t}$ el alineamiento Kabsch, $\mathcal{V}$ el conjunto válido, y $\mathcal{P}$ agrupa restricciones de simetría, límites, cruce de brazos y postura neutra. La expresión resume los términos del código; no añade observaciones LiDAR al ajuste articular. [38]

Con confianza $c_j \in [0, 1]$ se configura:

$$\sigma_j = 0.02 + (0.15 - 0.02)(1 - c_j), \qquad \pi_j = \sigma_j^{-2}, \qquad \Lambda_{dyn} = 0.25^{-2} I. \tag{22}$$

La interpretación probabilística de esta precisión es una modelización elegida: la confianza de la red no es, por sí misma, una varianza métrica calibrada. El prior dinámico se aplica por actualización con esa matriz fija; no debe atribuirse a este núcleo el mismo ruido por densidad temporal del EKF. [35, 38]

Se realizan dos pasos Gauss–Newton. El Jacobiano se obtiene mediante diferencias finitas centrales, de paso 10^{-3}, a través de cinemática y Kabsch. Para el residuo ya ponderado:

$$H \simeq \frac{J^T J}{M} + \Lambda_{dyn} + \lambda I + H_{neutral}, \qquad \boldsymbol{\mu} \leftarrow \boldsymbol{\mu} - H^{-1} \boldsymbol{g}, \qquad \Lambda \leftarrow H, \quad \lambda = 10^{-3}. \tag{23}$$

Algunos priors contribuyen solo al gradiente; no se usa el Hessiano exacto de todos los términos. El núcleo aplica además límites articulares y rechazo de innovaciones excesivas. Los términos internos de penalización de velocidad y aceleración están desactivados en el perfil revisado; un controlador de salida limita el movimiento del esqueleto publicado. [38, 35, 36]

7.3. Salida e incertidumbre

El esqueleto publicado usa límites de velocidad/aceleración y suavizado de la traslación y de la rotación mediante cuaterniones, con peso 0.2 a la pose nueva. La posición del nodo persona se toma de la pelvis. El seguimiento LiDAR modifica esta geometría ya ajustada; no estima ángulos de brazos a partir de la nube. [36, 39, 40]

La covarianza RT de la persona es una aproximación heurística:

$$\sigma^2_{\text{pos}} = \text{clip}(0.02\ \text{tr}(\Lambda^{-1}), 10^{-4}, 1). \tag{24}$$

Se asigna ese valor a las tres diagonales de posición y 1 a las de orientación. No es la propagación completa de profundidad, calibración, pose del robot y alineamiento global. Además, el valor llamado *free energy* en parte de la telemetría del agente se alimenta con `mean_l2`, un error medio en metros; no debe publicarse como si fuera el valor de $\mathcal{F}$ de la ecuación anterior. [39, 36]

8. Fusión LiDAR–ZED para seguimiento humano

8.1. Nacimiento y reparto de responsabilidades

La cámara inicia las hipótesis: `scaffold_missing_person_nodes` crea una persona desde un esqueleto con pelvis válida. El LiDAR recorre instancias ya existentes; no contiene en esta ruta un clasificador que cree personas nuevas a partir de clusters. Una persona que nunca ha visto la cámara puede producir ocupación geométrica, pero no obtiene por este mecanismo una identidad humana. [39, 40]

El LiDAR aporta dos funciones:

1. **Existencia**: retornos compatibles confirman; rayos que atraviesan el volumen esperado aportan evidencia contraria; oclusión o ausencia de rayos no aporta evidencia geométrica.
2. **Posición horizontal**: cuando no hay visión reciente, los puntos asociados desplazan el esqueleto conocido y la pista de identidad.

No hay una media ponderada simultánea de posiciones ZED y LiDAR con covarianza conjunta. La implementación es una **fusión de evidencia y seguimiento con prioridad visual y relevo geométrico**. [40, 41]

8.2. Selección de puntos y puerta espacial

Se seleccionan puntos en una banda global $z \in [1.20, 1.85]$ m. Esta banda pretende observar torso/cabeza por encima de las palmeras jóvenes. No sigue la anatomía de cada esqueleto ni estima automáticamente el suelo bajo cada trabajador. [40, 35]

Sea $r = 0.25$ m el radio corporal, ρ la distancia al sensor y Δt el tiempo desde la última posición actualizada. El código construye:

$$\sigma_{fix} = \begin{cases} 0.002\rho^2, & \text{última posición de cámara,} \\ 0.05, & \text{última posición de LiDAR,} \end{cases} \tag{25}$$

$$\sigma_{turn} = |\omega_{robot}|\, 0.05\, \rho, \tag{26}$$

$$\sigma = \text{mín}(\sigma_{fix} + \sigma_{turn} + 2.0\Delta t,\ 1.0), \tag{27}$$

$$g_L = \text{mín}(r + 2\sigma,\ 1.0). \tag{28}$$

Estas magnitudes son una envolvente heurística en metros; la suma lineal no es una propagación completa de covarianzas independientes. El término cuadrático con distancia modela incertidumbre de profundidad de cámara; el término de giro representa registro imperfecto durante latencia. [40, 35]

Cada punto de la banda se asigna a la persona más cercana cuya puerta lo admite. Esto evita que el mismo punto confirme simultáneamente dos centroides durante esta asignación, aunque no garantiza una identidad correcta si las puertas se solapan. Se admite incluso un solo punto para actualizar el centro, una decisión adaptada a la escasez de capas sobre una persona lejana. [40]

8.3. Actualización del centro

Si $\boldsymbol{m}$ es la media de puntos asociados y $\boldsymbol{o}$ el origen LiDAR en el plano, el centro propuesto es:

$$\boldsymbol{c}_L = \boldsymbol{m} + \frac{r}{2}\frac{\boldsymbol{m}-\boldsymbol{o}}{\|\boldsymbol{m}-\boldsymbol{o}\|}. \tag{29}$$

Se añade $r/2 = 0.125$ m hacia el interior porque los retornos representan la superficie visible. Esta corrección es geométrica y fija; un único punto lateral o una postura inusual puede sesgarla. [40]

Si la cámara se consideró observada desde el barrido previo, se conserva su centro y se registra la separación `lidar_cam_gap_m`. En caso contrario, se traslada en XY el esqueleto de salida, el último ajuste y su traslación suavizada. Se marca el fix como LiDAR y se llama a `refresh_track` para que la próxima detección visual pueda asociarse a la misma identidad. No se actualiza una velocidad humana latente ni una distribución cinemática de movimiento independiente. [40, 41, 34]

8.4. Fusión probabilística de existencia

La existencia se representa con log-odds:

$$L = \log\frac{P(E)}{1-P(E)}, \qquad P(E) = \frac{1}{1+e^{-L}}. \tag{30}$$

El nacimiento usa $L_0 = 2$ y se limita L al intervalo $[-4, 4]$. Los parámetros del modelo de detección son $p_D = 0.85$ y $p_C = 0.05$. Una observación de cámara aceptada añade:

$$a = \log\frac{p_D}{p_C} = \log 17 \simeq 2.833. \tag{31}$$

La cámara no incorpora en esta ruta una actualización negativa explícita por cada frame sin esqueleto; esa ausencia se trata a través del LiDAR y del tiempo sin evidencia. [40, 41, 42]

Para el LiDAR se intersectan rayos con una caja de torso de planta 0.5×0.5 m y la banda vertical indicada. El código separa evidencia de retorno superficial e_{occ}, retorno interior e_{int} y espacio atravesado e_{free}. En este modelo humano, tanto superficie como interior cuentan a favor. Un rayo detenido antes del volumen no demuestra ausencia. [40, 42]

La evidencia negativa se atenúa si el centro es incierto:

$$b = \log\frac{1-p_D}{1-p_C} \simeq -1.846, \tag{32}$$

$$w_{free} = \left(\frac{r}{r+\sigma_{carve}}\right)^2, \tag{33}$$

$$\Delta L_L = a\tanh\left(\frac{(e_{occ}+e_{int})a + w_{free}e_{free}b}{a}\right). \tag{34}$$

La saturación evita que muchos puntos correlacionados de un barrido cuenten como muchas confirmaciones independientes. Cuando no hay retornos asociados, σ_{carve} es la envolvente anterior; con centro LiDAR se usa 0.05 m más el término de giro. En la llamada a `carve_box`, la incertidumbre de superficie pasada es 0.05 m; la envolvente ampliada interviene después en w_{free}, no como un ensanchamiento idéntico de toda la geometría del ray tracing. [40]

En esta ruta no se configura la correlación entre frames de `ExistenceBelief`; su valor por defecto es cero. Por tanto, la saturación dentro del barrido sí está presente, pero no debe afirmarse que la aplicación humana descorrelaciona automáticamente toda evidencia temporal ni toda dependencia entre cámara y LiDAR. La creencia combina las aportaciones secuencialmente y aplica el límite tras cada integración. [42, 36, 40]

8.5. Persistencia y eliminación

Una persona se elimina si $P(E) < 0.2$, o si pasan más de 2 s sin evidencia de cámara ni barridos que alcancen su volumen. Este segundo mecanismo es un timeout explícito. Así, la oclusión mantiene el log-odds al no haber medición, pero **no garantiza persistencia indefinida**: el timeout puede eliminar la identidad. «No observación» y «espacio libre» no son equivalentes, aunque ambos puedan terminar retirando un nodo por mecanismos distintos. [41, 35]

El orden real de un ciclo es: leer esqueletos, crear nodos, preparar las observaciones, ejecutar el paso LiDAR sobre instancias existentes, ajustar/publicar cada persona y eliminar las ausentes. Por ello, la prueba de visión reciente del paso LiDAR se basa en marcas de ajustes anteriores; un esqueleto recién leído se incorpora al ajuste después de ese paso. Esta es otra razón para no describir el sistema como una actualización multimodal perfectamente simultánea. [41]

9. Fusión geométrica de obstáculos y rejilla residual

9.1. Ruta activa

El ciclo de `residual_concept` requiere un nuevo barrido LiDAR. Inicializa una rejilla de resolución 0.05 m a partir de los límites del campo, con margen, e integra el LiDAR y la nube obtenida de la profundidad ZED. Los rayos de cámara usan su propio origen, necesario para no atribuir a la cámara el espacio libre visible desde otro sensor. [44, 49, 45]

La ZED se restringe a profundidades de 0.3–5 m para esta ruta; el límite no afecta del mismo modo al esqueleto humano, cuya desproyección no comparte ese recorte de `ZedBoost`. Se submuestrea y voxeliza la nube de cámara, y se filtra infraestructura con una tolerancia que depende de distancia:

$$\sigma_{infra}(d) = 0.03 + 0.006d^2. \tag{35}$$

El ajuste del plano del suelo y el modelo de incertidumbre reducen la probabilidad de convertir suelo ruidoso en obstáculos. Con el Husky no existe Bpearl; la función de selección del suelo contiene un retorno alternativo al barrido disponible cuando no obtiene puntos Bpearl. [44, 45, 48]

9.2. Acumulación de evidencia

La rejilla acumula hits y evidencia de espacio libre por celda; al cerrar un ciclo se aplica una única actualización agregada, con precedencia del hit cuando hay conflicto. Tiene histéresis, memoria de estabilidad, representación de soporte en altura, tratamiento de la huella propia y mecanismos de olvido. El esquema conceptual $L_{i,k+1} = L_{i,k} + \Delta L_{i,k}$ no describe por sí solo todos esos factores; en particular, los pesos cambian con movimiento, alcance y certeza de pose. [49, 44]

Cuando está activada `Grid.PosePrecision`, un peso usado por la rejilla es de la forma:

$$w_{pose}(\rho) = \frac{s_{ref}^2}{s_{ref}^2 + \sigma_p^2 + \rho^2\sigma_\theta^2}. \tag{36}$$

Esto vincula explícitamente localización y mapeado: una pose incierta reduce la autoridad de los rayos para confirmar y borrar celdas. No convierte esos rayos en correcciones de pose del robot. [49, 44]

El producto activo es un nodo de rejilla con celdas y datos derivados para el controlador. Se excluyen explicaciones de infraestructura y objetos al leer los residuales, sin borrar por ello toda la evidencia sensorial subyacente. La semántica densa de suelo solo actúa si existe su producto; la etapa ADE20K está deshabilitada en la configuración Husky, aunque permanezcan opciones subordinadas de publicación con valor verdadero. [44, 30, 29]

9.3. Código conservado que no debe confundirse con la ruta activa

Existen clases de DBSCAN, ajuste de cajas residuales y la opción `ZedBoost.FeedDetection=true`. Sin embargo, en el `compute()` revisado la salida se obtiene de la rejilla; no se llama a la cadena antigua de creación y ajuste de instancias residuales. La variable `zed_detection_enabled` se carga pero no aparece utilizada en ese camino. Los comentarios de `residual_zed_boost.h` y de `observe()` que describen puntos añadidos a clusters o cajas son insuficientes para sostener que ese es el algoritmo activo. [44, 46, 48]

10. Cómo repercute la percepción en la navegación

El controlador toma la pose del campo, el polígono de trabajo, la rejilla residual, observaciones LiDAR y objetos del grafo. Su planificador de rejilla usa A* con comprobación de la huella del robot; el perfil tiene celda de 0.06 m y ruta continua. El modo de control seleccionado es `plain`, con `BandEnabled=true`; los comentarios heredados sobre MPPI no describen por sí solos el controlador configurado actualmente. Los límites declarados son 0.7 m/s de avance y 1.0 rad/s de giro. [50, 51, 52]

Cada nodo `person` añade un obstáculo poligonal de 12 lados que aproxima un disco de radio 0.8 m alrededor de su posición. El planificador sigue aplicando además la huella del robot. Esto permite evitar una persona incluso si su geometría no está bien representada en la banda de ocupación residual. El radio es un parámetro de evitación, no una distancia de seguridad certificada ni una predicción de la trayectoria humana. [53, 50]

Debe distinguirse **seguimiento perceptivo** de personas de **seguirlas con la base**. La ruta principal demostrada por el código revisado mantiene su identidad, publica postura/posición y las incorpora como obstáculos. El agente humano también propone acciones de observación valoradas mediante la precisión posterior del grado de libertad peor observado. Eso no demuestra la existencia de una política completa de acompañamiento de trabajadores ni de reconocimiento de su tarea agrícola. [41, 43, 1]

11. Parámetros esenciales para reproducir el perfil

Parámetro	Valor	Interpretación
Bridge: `Period.Compute`	8 ms	Paso solicitado de adquisición/actuación.
Bridge: `Period.Sensors`	50 ms	Muestreo solicitado de sensores pesados.
Bridge: `GPS.Period`	100 ms	Muestreo del receptor simulado.
Bridge: `FullPose.PublishPeriod`	100 ms	Publicación de telemetría de ruedas.
Openfield: `Period.Compute`	20 ms	Drena IMU, consulta GPS y publica pose.
Openfield: `GpsSigmaM`	0.05 m	Desviación de observación del EKF.
Openfield: `YawSigmaRad`	0.005 rad	Desviación de yaw absoluto.
Retina: `Period.Compute`	25 ms	Ciclo del agente; el worker tiene su propio coste.
Retina: `HumanPose.hold_ms`	500 ms	Caché de poses ante fallos transitorios.

Parámetro	Valor	Interpretación
Retina: `depth_patch`	2 px	Semiancho para muestrear profundidad.
Human: `Period.Compute`	50 ms	Ciclo de seguimiento.
Human: `BandZMin/Max`	1.20 / 1.85 m	Banda LiDAR global.
Human: `FollowGateMaxM`	1.0 m	Límite de puerta espacial.
Human: `DeathTimeoutS`	2.0 s	Tiempo máximo sin evidencia.
Residual: `Period.Compute`	100 ms	Ciclo con requisito de nuevo LiDAR.
Residual: `ZedBoost.MaxDepthM`	5.0 m	Recorte de profundidad para ocupación.
Controller: `ControlMode`	`plain`	Seguimiento seleccionado.
Controller: `PersonRadiusM`	0.8 m	Radio del obstáculo humano.

Fuentes: perfiles activos. Las frecuencias inversas de estos períodos son nominales. El paso Webots, la carga gráfica, la planificación de hilos y el coste ONNX pueden reducir la frecuencia efectiva. El driver ZED configurado a 30 FPS no crea 30 frames nuevos por segundo si el productor simulado entrega menos. [7, 27, 29, 35, 45, 50, 16]

12. Limitaciones y hallazgos que deben figurar en el reporte

12.1. No existe fusión LiDAR–ZED en el EKF del Husky

La localización activa depende del GPS y del yaw absoluto para corregir deriva. No se observó scan matching, corrección por landmarks visuales, odometría estéreo ni cierre de bucle conectados a `OpenFieldEkf`. La disponibilidad de LiDAR y cámara no equivale a usarlos para estimar la pose. [24, 26]

12.2. Modelo planar en un terreno tridimensional

El terreno puede tener camellones e irregularidades, pero la pose global publicada es SE(2), con altura y roll/pitch nulos en la arista dinámica. Si el robot se inclina, los montajes estáticos y el yaw no bastan para registrar perfectamente la nube y la profundidad en 3D. El efecto sobre la banda del torso y el suelo es una hipótesis geométrica que requiere medición, no un fallo cuantificado en esta revisión. [5, 28]

12.3. Reutilización de poses antiguas como si fueran nuevas

`YoloHumanProcessor::detect_poses` conserva `last_poses_` hasta 500 ms después de la última detección si el detector no devuelve personas. `PoseStage::run` marca `poses_fresh=run_now`, que significa «se ejecutó el detector», no «se obtuvo una detección nueva». El agente publica después esos keypoints usando la profundidad y el timestamp del bundle actual. [32, 33, 30]

Por tanto, en una pérdida transitoria de detección, una pose 2D retenida puede volver a desproyectarse con profundidad nueva y contarse como evidencia visual reciente. Esta posibilidad contradice la lectura literal de «retención solo para el visor» de algunos comentarios. No se midió su frecuencia en ejecución. Una corrección futura debería separar datos retenidos para visualización de observaciones con adquisición real nueva. [31, 41]

12.4. Transformación ausente en el consumidor humano

Si `DsrSkeletonSource::poll` no obtiene `field<-zed`, utiliza identidad y devuelve puntos de cámara; escribe un aviso, pero no descarta necesariamente el cuerpo. Si el campo existe pero falla la cadena temporal RT, esos puntos podrían acabar interpretados como coordenadas globales por el creador de personas. Es un riesgo de registro visible en el código; no se reproduce aquí un caso concreto. [34, 39, 41]

12.5. Asincronía y reutilización de profundidad en ocupación

`integrate_zed_into_grid` llama a `pump()` pero comprueba `has_depth()`, sin exigir que la llamada haya devuelto un frame nuevo. Además, consulta la transformación de cámara con `current_ts_`, el timestamp del LiDAR, en lugar de emplear explícitamente el de profundidad. Una profundidad retenida puede reintegrarse al llegar otros barridos; con movimiento o retraso entre sensores puede introducir evidencia repetida o registro desplazado. No debe afirmarse que esta ruta posee la misma sincronización RGB-D de 20 ms de retina. [44, 47]

12.6. Supuestos del seguimiento LiDAR

La banda 1.20–1.85 m y la aceptación de un solo punto explotan un escenario con vegetación baja. Ramas, árboles, vehículos u objetos altos pueden producir retornos compatibles. Un trabajador agachado puede salir de la banda, y un terreno elevado altera su altura global. La clasificación semántica inicial procede de la cámara; las confirmaciones LiDAR no son un clasificador humano generalizable sin validación. La puerta máxima de 1 m y el timeout de 2 s también limitan las pérdidas tolerables. [40, 35, 5]

12.7. Incertidumbres parciales y dependencia de la simulación

La covarianza de persona es un proxy; la puerta LiDAR es una envolvente heurística; el ajuste humano convierte confianza de red a ruido métrico por una regla elegida. No existe un posterior conjunto que incluya estado del robot, extrínsecas, IDs, articulaciones y todas sus correlaciones. Tampoco la emulación RGB-D reproduce por sí misma fallos reales de correspondencia estéreo. Estas limitaciones impiden trasladar directamente resultados simulados a una ZED y un LiDAR físicos. [39, 40, 38, 17]

12.8. GPS retrasado y odometría retenida

La interfaz GPS utilizada no entrega timestamp del fix; el filtro aplica la corrección al estado tras drenar IMU. No se observó una actualización retardada con rebobinado del estado. Asimismo, `read_odometry` conserva la última velocidad si no hay atributo nuevo, y `predict` sigue usándola mientras llega IMU. El timeout primario del agente vigila IMU; el tratamiento de pérdidas de ruedas y retrasos GPS debe ensayarse separadamente. [26, 27]

12.9. Documentación y opciones heredadas

El README del núcleo humano todavía presenta como pendiente la integración con DSR, aunque esa integración sí existe. La configuración LiDAR cita un alcance heredado; `FeedDetection` no demuestra una ruta activa de clusters; el perfil del controlador mantiene comentarios MPPI con modo `plain`. Por ello, las afirmaciones de este informe se apoyan en llamadas ejecutables además de documentación. [54, 14, 44, 50]

13. Validación realizada y evaluación propuesta

13.1. Comprobaciones ejecutadas durante la revisión

Se compiló y ejecutó el `self_test()` existente de `OpenFieldEkf`, aislado del resto del sistema. La prueba genera 20 000 pasos a 8 ms, equivalentes a 160 s de movimiento sintético, con tramos rectos, pivotes, arcos y paradas. Tras los primeros 5 s, comprueba error máximo de posición inferior a cinco desviaciones GPS, error máximo de yaw inferior a 1° y positividad de la covarianza final. [24]

Resultado del test sintético	Valor impreso
Estado	PASS
Máximo error de posición	0.035 m
Máximo error de yaw	0.317°
Covarianza final definida positiva	Sí
Desviación final horizontal agregada	0.011 m
Desviación final de yaw	0.0017 rad

Estos son resultados redondeados de esa secuencia artificial con semilla fija, **no** una precisión medida en el campo Webots. El test no reproduce toda la asincronía de procesos, la correlación GPS configurada en el PROTO o el terreno. Incluso sus comentarios de tasas deben interpretarse con cuidado: el bucle aplica GPS y yaw cada 12 pasos en fases distintas; a 8 ms son intervalos de 96 ms para cada uno. [24]

También se compiló el núcleo humano independiente de DSR/Qt y se ejecutó su `sanity_checks` mediante CTest: **1/1 prueba pasó**. Verifica propiedades básicas de cinemática, Kabsch y ajuste de una postura sintética sin ruido; no certifica detección YOLO, seguimiento de IDs ni fusión LiDAR. No se ejecutó la prueba de paridad con el antiguo prototipo Python. [56, 55]

No se lanzó el stack sensoriomotor ni se mandaron movimientos al robot. Tampoco se generaron medidas nuevas de precisión/recall, error por articulación, tasa de cambio de identidad, latencia global o colisiones. El informe es principalmente una auditoría del flujo de implementación, complementada por las dos pruebas aisladas anteriores.

13.2. Protocolo recomendado para un reporte experimental

Las siguientes pruebas son propuestas, no resultados:

1. **Localización**: repetir trayectorias rectas, giros sobre sí mismo y arcos en los mundos plano y de campo. Comparar EKF completo, GPS desactivado y yaw desactivado mediante los interruptores existentes. Registrar RMSE, percentiles y máximos de posición/yaw contra la referencia temporalmente alineada.
2. **Consistencia**: analizar innovaciones GPS y, si se instrumentan, NIS/NEES. Un error pequeño no demuestra que P esté bien calibrada. Inyectar retardos y pérdidas independientes de GPS, ruedas e IMU.
3. **Ablación humana**: comparar `HumanLidar.Enabled=true/false` con la misma trayectoria. Medir duración de pistas, cambios de identidad, falsos nacimientos, pérdidas al salir del campo visual y error del centro frente a trayectorias de referencia.
4. **Escenarios difíciles**: cruce de dos trabajadores, persona agachada, oclusiones, giro del robot y rangos crecientes. Separar «persona nunca detectada» de «persona conocida mantenida por LiDAR».
5. **Timestamps**: registrar edades RGB, profundidad, LiDAR, pose y publicación de persona; comprobar la retención de keypoints y profundidad descrita en la sección 12.

6. **Ocupación**: comparar rejilla LiDAR sola con LiDAR más ZED, desactivando `ZedBoost.FeedGrid` para la primera condición. Medir falsos obstáculos de suelo, cobertura, tiempo de borrado y efecto sobre rutas.
7. **Transferencia**: antes de ensayar hardware, verificar unidades, extrínsecas y fuente del yaw absoluto; después evaluar con escenas y oclusiones reales. Los peatones sin colisión física no bastan como evaluación de seguridad.

Para reproducibilidad deben guardarse el mundo, configuraciones, pesos ONNX y sus hashes, versión del árbol de trabajo, semilla cuando exista, factor de tiempo real, hardware de inferencia y timestamps. Los CSV existentes de localización, ajuste humano y ocupación aportan parte de esta evidencia; los IDs y la pose de referencia humana necesitarían comprobarse o instrumentarse para una evaluación completa. [26, 41, 44]

14. Construcción y ejecución del proyecto

El proyecto requiere dependencias externas: RoboComp, Cortex/DSR con el tipo `field`, Webots, Eigen, Qt, FastDDS, ONNX Runtime y los modelos solicitados por retina. El README especifica compilación por componente y limita el paralelismo a ocho trabajos. No hay una compilación monolítica del repositorio que sustituya a todas esas instalaciones. [1]

Los pasos documentados para el perfil simulado son:

```
# En cada componente que deba construirse:
cmake -B build
make -C build -j8

# Con IceStorm disponible y el mundo Husky abierto en Webots:
cd active_inference/utils
python3 subcognitive.py sub_husky.toml

# En otra terminal, desde el mismo directorio:
python3 cognitive.py cognitive_husky.toml

# Objetivo de ejemplo en coordenadas del campo; requiere
# habilitar el control mediante Run en el perfil actual:
python3 send_goal.py X Y
```

Los lanzadores contienen rutas basadas en `~/robocomp/AiGB_UEx_Navigation`. Los modelos ONNX no se versionan según el README. La ausencia de un modelo, del descriptor DDS o de una transformación puede desactivar etapas o impedir resultados aun cuando el proceso se haya iniciado. [2, 3, 1, 33]

El presente documento es autocontenido y no requiere archivo bibliográfico externo:

```
cd docs
pdflatex -interaction=nonstopmode -halt-on-error informe_fusion_sensorial.tex
pdflatex -interaction=nonstopmode -halt-on-error informe_fusion_sensorial.tex
```

15. Redacción sintética utilizable en un technical report

El sistema desarrollado organiza la percepción y navegación mediante agentes RoboComp que comparten un grafo DSR. La localización del Husky A300 se obtiene con un EKF planar cuya predicción combina velocidad de ruedas y velocidad angular de IMU, y cuyas correcciones utilizan GPS y yaw absoluto. La observación GPS incorpora el desplazamiento de la antena respecto al origen corporal. La pose y su covarianza permiten expresar datos sensoriales en el marco global del campo.

La detección humana parte de keypoints 2D obtenidos por YOLO-pose en la imagen ZED; su profundidad local permite reconstruir un esqueleto BODY_18 tridimensional. Después del registro en el campo y la asociación temporal, un modelo cinemático de 11 grados de libertad ajusta la postura mediante pasos Gauss–Newton y una aproximación de Laplace. El LiDAR complementa a la visión sobre personas ya detectadas: confirma o refuta su existencia mediante rayos en una banda de torso, y actualiza su centro horizontal cuando la cámara pierde observación reciente. Las aportaciones de existencia se combinan en log-odds y la posición se actualiza mediante prioridad visual y relevo LiDAR, sin un filtro articular multimodal único.

En paralelo, una rejilla residual integra LiDAR y profundidad RGB-D para representar obstáculos. El controlador combina esa ocupación con la pose del robot y obstáculos derivados de las personas. En el perfil simulado, la profundidad se obtiene de un RangeFinder de Webots; no se ha demostrado aquí la equivalencia de rendimiento con una ZED física. La arquitectura implementa por tanto localización GPS–inercial– odométrica y percepción geométrica/semántica complementaria, con limitaciones de sincronización, asociación e incertidumbre que deben evaluarse experimentalmente.

Las fuentes siguientes son código y configuraciones locales consultados el 14 de octubre de 2026; no son referencias bibliográficas externas. Los localizadores se refieren al árbol de trabajo analizado, y las huellas muestran los primeros 16 dígitos hexadecimales de SHA-256. Un archivo conservado no implica que todas sus rutas estén activas.

Referencias

[1] Descripción general, dependencias y lanzamiento. `README.md`. Archivo completo (86 líneas). Huella SHA-256 abreviada: `e53ac2a50fbf1131`.

[2] Perfil de lanzamiento sensoriomotor. `active_inference/utils/sub_husky.toml`. Archivo completo (46 líneas). Huella SHA-256 abreviada: `131d62e3696e99a4`.

[3] Perfil de lanzamiento cognitivo. `active_inference/utils/cognitive_husky.toml`. Archivo completo (43 líneas). Huella SHA-256 abreviada: `4c793a6f79160d90`.

[4] Selección del marco global. `active_inference/common/world_frame/world_frame.h`. Archivo completo (88 líneas). Huella SHA-256 abreviada: `91a84e75fbc4842c`.

[5] Mundo agrícola y peatones; véanse también husky_test.wbt y husky_apple.wbt. `webots-agri/worlds/husky_field.wbt`. Localizadores de inicio: líneas 15, 56. Huella SHA-256 abreviada: `160086b558d88a63`.

[6] Geometría y sensores simulados. `webots-agri/protos/HuskyA300.proto`. Localizadores de inicio: líneas 39, 135, 167. Huella SHA-256 abreviada: `eb9de552ab1e018f`.

[7] Perfil Husky del bridge. `webots-bridge/etc/config_husky`. Archivo completo (203 líneas). Huella SHA-256 abreviada: `d96ec91601d80de3`.

[8] Modelo DSR del robot y extrínsecas. `active_inference/robot_concept/husky.json`. Archivo completo (494 líneas). Huella SHA-256 abreviada: `8843802c663bbd2e`.

[9] Perfil del productor del grafo. `active_inference/robot_concept/etc/config_husky.toml`. Archivo completo (240 líneas). Huella SHA-256 abreviada: `57b1261263ec2843`.

[10] Telemetría de ruedas y referencia del supervisor. `active_inference/robot_concept/src/specificworker.cpp`. Localizadores de inicio: líneas 2126. Huella SHA-256 abreviada: `c67af76ee10bd696`.

[11] Adquisición de RGB y profundidad. `webots-bridge/src/specificworker.cpp`. Localizadores de inicio: líneas 361. Huella SHA-256 abreviada: `6940a92fced0e74a`.

[12] Conversión y consulta GPS. `webots-bridge/src/specificworker.cpp`. Localizadores de inicio: líneas 2123, 2180, 2225. Huella SHA-256 abreviada: `6940a92fced0e74a`.

[13] Lectura de IMU del simulador. `webots-bridge/src/specificworker.cpp`. Localizadores de inicio: líneas 1963. Huella SHA-256 abreviada: `6940a92fced0e74a`.

[14] Montaje, filtro y descriptor LiDAR. `robocomp-robolab/components/hardware/laser/lidar3d_dds/etc/config_helios_husky_webots.toml`. Archivo completo (277 líneas). Huella SHA-256 abreviada: `1fc2163bc7e3b70b`.

[15] Conversión de nube, autofiltro y publicación. `robocomp-robolab/components/hardware/laser/lidar3d_dds/src/specificworker.cpp`. Localizadores de inicio: líneas 419, 601, 695. Huella SHA-256 abreviada: `ecd4c5ef8eb85867`.

[16] Perfil ZED simulado. robocomp-robolab/components/hardware/camera/zed_camera/etc/configHuskyWebot.toml. Archivo completo (58 líneas). Huella SHA-256 abreviada: 44333298e7a24157.

[17] Ramas SDK/simulación, profundidad y publicación de pose. robocomp-robolab/components/hardware/camera/zed_camera/src/specificworker.cpp. Localizadores de inicio: líneas 205, 342, 483, 600, 660. Huella SHA-256 abreviada: 7d70f57475feb156.

[18] Formato de profundidad DDS. robocomp-robolab/components/hardware/camera/zed_camera/src/dds_publisher.cpp. Localizadores de inicio: líneas 124. Huella SHA-256 abreviada: 9761d4b518916920.

[19] Descriptores, tópicos y fábricas de suscriptores. active_inference/common/media_transport/media_transport.h. Localizadores de inicio: líneas 466, 503. Huella SHA-256 abreviada: d17b5e7d00bfa0df.

[20] Lectura de barridos y transformación al instante de captura. active_inference/common/media_transport/lidar_plane_reader.cpp. Localizadores de inicio: líneas 134, 161, 191. Huella SHA-256 abreviada: 7b084b17b8b4bbed.

[21] Unidades de profundidad y emparejamiento RGB-D. active_inference/retina/src/media_plane_source.cpp. Localizadores de inicio: líneas 163, 442, 458. Huella SHA-256 abreviada: 5312aebccd09b6b6.

[22] Buffer temporal de RGB y profundidad. active_inference/retina/src/zed_frame_aligner.cpp. Localizadores de inicio: líneas 34. Huella SHA-256 abreviada: 816702efa93783fc.

[23] Selección del frame más nuevo y pose de cámara. active_inference/retina/src/zed_source.cpp. Localizadores de inicio: líneas 26. Huella SHA-256 abreviada: 98a3361b9490d4e2.

[24] Inicialización, ecuaciones EKF y prueba sintética. active_inference/openfield_concept/src/openfield_ekf.cpp. Localizadores de inicio: líneas 32, 48, 85, 109, 129. Huella SHA-256 abreviada: e28bce92da1d9a80.

[25] Estado, marcos y parámetros del EKF. active_inference/openfield_concept/src/openfield_ekf.h. Localizadores de inicio: líneas 31, 51. Huella SHA-256 abreviada: 71c2c5aa9f6d3b57.

[26] Orquestación de sensores y evaluación contra referencia. active_inference/openfield_concept/src/specificworker.cpp. Localizadores de inicio: líneas 203, 223, 278, 297, 380. Huella SHA-256 abreviada: 2c82d557713c7fbd.

[27] Parámetros de localización y campo. active_inference/openfield_concept/etc/config.toml. Archivo completo (86 líneas). Huella SHA-256 abreviada: 5f09dd9efa600ab3.

[28] Creación de field, inversión de pose y publicación RT. active_inference/openfield_concept/src/openfield_scene_graph.cpp. Localizadores de inicio: líneas 24, 70. Huella SHA-256 abreviada: f425aceb1fb14f0c.

[29] Modelos y etapas visuales del perfil Husky. active_inference/retina/etc/config_husky.toml. Archivo completo (541 líneas). Huella SHA-256 abreviada: f5c1c80dbe469ff1.

[30] Construcción de etapas y publicación de resultados. active_inference/retina/src/specificworker.cpp. Localizadores de inicio: líneas 193, 209, 1716. Huella SHA-256 abreviada: 3579c370351fe4f1.

[31] Publicación y desproyección BODY 18. active_inference/retina/src/graph_publisher.cpp. Localizadores de inicio: líneas 911, 925. Huella SHA-256 abreviada: 85e5be6c00e75118.

[32] Detector y retención de poses. `active_inference/retina/src/yolo_human.cpp`. Localizadores de inicio: líneas 283. Huella SHA-256 abreviada: `ba649483bab297e7`.

[33] Etapa de pose y semántica de poses_fresh. `active_inference/retina/src/pose_stage.cpp`. Localizadores de inicio: líneas 24. Huella SHA-256 abreviada: `2d470d065959c32b`.

[34] Registro, asociación y refresco de IDs. `active_inference/human_concept/src/skeleton_source.cpp`. Localizadores de inicio: líneas 108, 225, 244. Huella SHA-256 abreviada: `b7d6ae1731ed2d61`.

[35] Parámetros humanos y LiDAR. `active_inference/human_concept/etc/config.toml`. Archivo completo (106 líneas). Huella SHA-256 abreviada: `c86def28a08049d4`.

[36] Instancias, observación, inferencia y suavizado. `active_inference/human_concept/src/human_fitter.cpp`. Localizadores de inicio: líneas 75, 107, 136. Huella SHA-256 abreviada: `aaa498cc3cd09936`.

[37] Modelo BODY 18 y definición de 11 grados de libertad. `active_inference/human_concept/cpp/core/human_kinematic_model.h`. Localizadores de inicio: líneas 17, 109. Huella SHA-256 abreviada: `fadbc7d0da7adb09`.

[38] Kabsch, precisiones, Gauss–Newton y aproximación de Laplace. `active_inference/human_concept/cpp/core/vfe_inference.cpp`. Localizadores de inicio: líneas 23, 208. Huella SHA-256 abreviada: `3690da77f58fe1d7`.

[39] Nacimiento de personas, esqueleto y covarianza proxy. `active_inference/human_concept/src/human_scene_graph.cpp`. Localizadores de inicio: líneas 52, 122. Huella SHA-256 abreviada: `234a11a833e18dde`.

[40] Fusión de existencia y seguimiento geométrico. `active_inference/human_concept/src/human_lidar_presence.cpp`. Localizadores de inicio: líneas 52, 62. Huella SHA-256 abreviada: `bd7dfda4caa528e6`.

[41] Orden de actualización, eliminación y acciones de observación. `active_inference/human_concept/src/specificworker.cpp`. Localizadores de inicio: líneas 250, 299, 327, 358, 416. Huella SHA-256 abreviada: `be3bf9a780f5d5a8`.

[42] Ray casting, saturación y creencia de existencia. `active_inference/common/existence_belief/existence_belief.h`. Localizadores de inicio: líneas 103, 112, 392, 499. Huella SHA-256 abreviada: `63d39fb3dee8e749`.

[43] Propuesta de vista y ganancia esperada de información. `active_inference/human_concept/src/epistemic_planner.cpp`. Localizadores de inicio: líneas 16. Huella SHA-256 abreviada: `fa80c3a6471a327f`.

[44] Ruta activa de ocupación, integración ZED y código residual conservado. `active_inference/residual_concept/src/specificworker.cpp`. Localizadores de inicio: líneas 672, 1115, 1378, 1828. Huella SHA-256 abreviada: `f9f5a039acd83f22`.

[45] Perfil de rejilla y profundidad. `active_inference/residual_concept/etc/config_husky.toml`. Archivo completo (519 líneas). Huella SHA-256 abreviada: `abdec61d37361854`.

[46] Carga de FeedDetection y otras opciones. `active_inference/residual_concept/src/residual_config.cpp`. Localizadores de inicio: líneas 104. Huella SHA-256 abreviada: `a61e0c8a6a6c2cdb`.

[47] Retención y unidades del frame de profundidad. `active_inference/residual_concept/src/residual_zed_ingestor.cpp`. Localizadores de inicio: líneas 69. Huella SHA-256 abreviada: `af5655569a0a1b01`.

[48] Geometría de desproyección y límites de nube ZED. `active_inference/residual_concept/src/residual_zed_boost.h`. Localizadores de inicio: líneas 37, 63. Huella SHA-256 abreviada: `eb3d46a0b418088d`.

[49] Integración de rayos, precisión de pose y consolidación de celdas; parámetros en el .h homónimo. `active_inference/residual_concept/src/residual_occupancy_grid.cpp`. Localizadores de inicio: líneas 486, 263, 524. Huella SHA-256 abreviada: `3338460e7f8fbfd8`.

[50] Modo plain, ruta, límites y obstáculos humanos. `active_inference/controller/etc/config_husky.toml`. Localizadores de inicio: líneas 122, 289, 533, 555. Huella SHA-256 abreviada: `e7084e2b78bf44d8`.

[51] Planificación de rejilla y comprobación de huella. `active_inference/controller/src/grid_planner.cpp`. Localizadores de inicio: líneas 653. Huella SHA-256 abreviada: `fef90402ef02dfdb`.

[52] Seguimiento y modificación local de trayectoria. `active_inference/controller/src/trajectory_controller.cpp`. Archivo completo (1347 líneas). Huella SHA-256 abreviada: `37ea2982f25cbca5`.

[53] Obstáculos derivados de nodos person. `active_inference/controller/src/controller_obstacle_tracker.cpp`. Localizadores de inicio: líneas 1180. Huella SHA-256 abreviada: `9e58ff7aedab5e69`.

[54] Documentación del núcleo y tareas históricas pendientes. `active_inference/human_concept/cpp/README.md`. Archivo completo (57 líneas). Huella SHA-256 abreviada: `13697f290e9b654d`.

[55] Construcción aislada y registro de sanity_checks. `active_inference/human_concept/cpp/CMakeLists.txt`. Localizadores de inicio: líneas 32. Huella SHA-256 abreviada: `ccf38a57d46989b6`.

[56] Comprobaciones de cinemática, Kabsch e inferencia sintética. `active_inference/human_concept/cpp/test/sanity_checks.cpp`. Localizadores de inicio: líneas 28. Huella SHA-256 abreviada: `90382a4d1ed33025`.